WorkoutLang

CSE 341 — Concepts of Programming Languages | Gebze Technical University

D4: AI Usage Journal (Part 2)

Student: 210104004053 | Part 2 entries: 9 May – 22 May 2026 | Tool: Claude Sonnet 4.6

This journal documents my substantive interactions with AI assistants during Part 2 of the WorkoutLang project. Each entry records the goal, the exact prompt sent, the relevant portion of the response, what I accepted or rejected, any errors I caught, and a reflection. Entries are ordered chronologically. Entries 9 of 11 and 10 of 11 are the mandatory experiments E2 and E3.

Entry 6 of 11 · 2026-05-17 · Type checker — design · Claude Sonnet 4.6

Goal	I needed to decide whether to keep the type checker's scope state in its own SymbolTable copy or share the one the interpreter would later use, and I wanted to think through the consequences before writing any code.
Prompt	I have a WorkoutLang type checker that needs to walk the AST in two passes. Pass 1 registers global names (workout, routine, func). Pass 2 walks bodies and checks types. Should the type checker use its own SymbolTable instance or share the interpreter's? The symbol table uses an environment chain (linked list of scope frames). Types: int, float, bool, duration, weight, Day.
Response (key part)	The AI said: use a separate instance. Its main argument was that the type checker stores type information only — it does not need runtime values — so sharing state with the interpreter would mean the interpreter inherits type-checker scope frames that were never properly cleaned up. It also pointed out that if type checking fails, the interpreter should never run, so there is no benefit to sharing. It suggested storing function signatures in a flat array parallel to the symbol table rather than in the symbol table itself, since the symbol table cannot hold arity and parameter types.
Accepted	Separate instances — correct reasoning. The parallel FuncSig array for function signatures was the most useful concrete suggestion; I adopted it directly as tc->funcs[] in type_checker.h.
Rejected / Modified	The AI suggested a third pass to check that every func has a return statement. I dropped this — it requires control-flow analysis, which is beyond the scope of D1 §4.5, and the spec acknowledges the gap. Adding a broken third pass that only handles the trivial case would be worse than documenting the limitation honestly.
Errors you caught	The AI said 'the symbol table can store WL_Type directly in the Symbol struct.' This is true — WL_Type is already a field in Symbol. But it then suggested using a second symbol table to store parameter types for functions. That was unnecessary given the FuncSig array it had just recommended. The two suggestions contradicted each other. I combined them into the single FuncSig array approach.
Reflection	Asking an architectural question before writing code was more useful than asking for code and then trying to restructure it. The AI reasoned well about separation of concerns but produced a slight inconsistency when it started elaborating — the first answer was better than the elaboration. In future I'll ask a follow-up only when the first answer is clearly incomplete, not just to get more detail.

Entry 7 of 11 · 2026-05-18 · Interpreter — design · Claude Sonnet 4.6

Goal	I was writing the interpreter's function call handler and wasn't sure whether to evaluate arguments before or after opening the callee's scope frame — the order matters for call-by-value semantics.
Prompt	In a tree-walking interpreter implementing call-by-value parameter passing (Sebesta §9.2), when a function call EXPR_CALL is evaluated, at what point should I evaluate the argument expressions — before or after opening the new SCOPE_FUNCTION frame? My scope frames use an environment chain. The callee must get copies of the argument values, not references to the caller's variables.
Response (key part)	The AI gave the correct answer: evaluate all arguments in the caller's scope first, store the results in a temporary array, then open the callee's scope frame and bind each parameter to its stored value. It said: if you evaluate arguments after opening the new frame, the argument expressions are evaluated in the callee's (empty) environment, so any variable references in the arguments would fail with 'undefined variable.' It also noted that this is exactly what the C calling convention does: arguments are evaluated and pushed to the stack before the return address and new frame are set up.
Accepted	The evaluate-before-open pattern directly. In interpreter.c, EXPR_CALL now fills argv[] in a loop over e->call.args before the scope_enter call. The C calling convention analogy was a useful mental model.
Rejected / Modified	The AI suggested using a dynamically allocated array for argv to handle arbitrary argument counts. WorkoutLang has a fixed parameter limit of 16 (TC_MAX_PARAMS in type_checker.h), so a stack-allocated array of 16 RtVal values is sufficient and avoids malloc. I used int argv[16] instead.
Errors you caught	None in the main answer. The dynamic allocation suggestion was not an error — it was the general-purpose approach — but it was unnecessary for WorkoutLang's fixed limit. The AI did not know my type checker already enforced the cap.
Reflection	This session confirmed something from Entry 3 (Part 1): giving the AI the relevant constraint upfront ('fixed limit of 16 parameters') produces a more targeted answer than accepting the general solution and trimming it down. The question was well-posed and the answer was immediately usable — this is what AI is good at: explaining ordering decisions in data structures where the trade-offs are known but the implementation detail is fiddly.

Entry 8 of 11 · 2026-05-18 · Writing — D1 §4.4 semantics · Claude Sonnet 4.6

Goal	My D1 §4.4 S-Set semantic rule concludes with σ[log ↦ σ.log ++ {ex, n, m, d}] but the interpreter calls printf directly — I needed a sentence to bridge this without changing either the spec or the implementation.
Prompt	In my D1 §4.4 I have this S-Set conclusion: σ[log ↦ σ.log ++ {ex, n, m, d}]. But my C interpreter does not maintain an in-memory log — it calls printf directly to stdout. This creates a contradiction between the spec and the implementation. Write one sentence to add immediately after the S-Set conclusion that resolves this. The sentence should say that σ.log is an abstraction over the I/O channel and the observable effect is identical. It must be defensible if an examiner asks about it.
Response (key part)	The AI produced: 'In the implementation, σ.log is realised as the standard output channel rather than an in-memory sequence: each application of S-Set writes exactly one entry to stdout, producing the same observable effect as an append to σ.log — the abstraction over the I/O channel is justified because WorkoutLang has no operation that reads back from the log, so no program can distinguish an in-memory accumulation from a write-through to output.'
Accepted	The sentence almost verbatim. The core argument — no read-back operation means the two realisations are observationally equivalent — is correct and is the standard justification for I/O abstraction in operational semantics. I kept it and added it to §4.4 immediately after the S-Set rule.
Rejected / Modified	The AI offered a longer version that added a footnote reference to denotational semantics treatment of I/O. WorkoutLang uses operational not denotational semantics (§4.4 explicitly states this), so the footnote would introduce a style inconsistency. Dropped it.
Errors you caught	The AI initially said 'no program can observe the log at all.' That is slightly too strong — a program could observe side effects through timing, for instance. The correct claim is narrower: WorkoutLang has no language-level operation to read from the log. I rewrote that clause to be precise. If an examiner asked 'could you in principle distinguish the two realisations?' the weaker original phrasing would have been indefensible.
Reflection	I gave the AI the conclusion I wanted to reach ('observable effect is identical') in the prompt, which made the output immediately useful rather than requiring me to argue for the framing first. The precision error — 'cannot observe at all' vs 'no language-level read-back operation' — is exactly the kind of subtle overclaim that causes trouble under exam questioning. Checking AI output against edge cases ('what if an examiner pushes on this?') is worth doing before copying anything into a spec.

Entry 9 of 11 — Experiment E2 · 2026-05-19 · Writing — D1 §4.5 type system · Claude Sonnet 4.6

Goal	To verify my understanding of name vs structural equivalence for the Day record type (setCount: int field) against Sebesta §6.14, and to check whether the AI's example would actually run under the rule it described.
Prompt	WorkoutLang has one record type called Day with a single field: setCount of type int. Day values are declared with the 'day' keyword inside workout blocks and are immutable. I chose name equivalence for Day (Sebesta §6.14). Explain the difference between name equivalence and structural equivalence for this specific record type. Give a concrete WorkoutLang code example showing a case where the two rules would give different results. Does your explanation match Sebesta §6.14?
Response (key part)	The AI explained correctly that under name equivalence, two types are compatible only if they share the same declared name. Under structural equivalence, two types are compatible if their fields match in name, type, and order. It gave a LegDay/ArmDay example, then extended to say: 'More importantly for WorkoutLang, duration and weight are both internally stored as numbers. Under structural equivalence they could be confused. Name equivalence prevents this.'
Accepted	The duration/weight observation was the most useful part — it made the consequence concrete for WorkoutLang's existing types, not just the Day record. I used this argument in the D1 §4.8 name equivalence rationale paragraph.
Rejected / Modified	The LegDay/ArmDay example the AI gave is wrong for WorkoutLang specifically. LegDay and ArmDay are the same type (Day) — they are both instances of the Day record type, not separate type declarations. WorkoutLang does not allow the user to declare a new record type; there is only one: Day. So the example does not illustrate name-vs-structural equivalence for record types. I did not use it.
Errors you caught	Two. First, the LegDay/ArmDay type confusion above — two values of the same type are not two different types. Second, the AI said 'Sebesta §6.14 defines name equivalence as requiring identical names at every level of nesting.' Checked Sebesta: §6.14 distinguishes declaration equivalence from name equivalence and notes that different languages implement different variants. The AI's phrasing was a simplification. I kept the §6.14 citation but used Sebesta's own terms, not the AI's paraphrase.
Reflection	The E2 experiment confirmed that the AI understands the distinction abstractly but does not model WorkoutLang's specific type system well enough to generate a correct domain example without prompting. The best output came from the duration/weight observation — which the AI added on its own — not from the direct answer to my question. AI is better at generating related observations than at constructing correct examples for a language it has only seen described in a prompt.

Entry 10 of 11 — Experiment E3 · 2026-05-19 · Type checker — implementation · Claude Sonnet 4.6

Goal	To get the AI to implement the STMT_VAR_DECL and STMT_ASSIGN type-checking handlers in C, then test them on three programs I had already run by hand to know the expected output.
Prompt	Here is my type_checker.h [pasted]. Write the C code for the STMT_VAR_DECL and STMT_ASSIGN cases in tc_check_stmt(). Rules: (1) STMT_VAR_DECL — the declared type must exactly match the initialiser expression type; no coercion. If mismatch, emit [line N] Type error: cannot initialise '<name>' (type X) with expression of type Y. (2) STMT_ASSIGN — the target must be a mutable variable in scope. Assigning to a workout/routine/func name is a type error. Type mismatch is also a type error. No implicit coercion.

Response (key part)	`TC_Type declared = tc_type_from_name(s->var_decl.type_name);`
	`TC_Type init_type = tc_check_expr(tc, s->var_decl.init);`
	`if (init_type != TC_TYPE_UNKNOWN && init_type != declared) {`
	`  snprintf(buf, sizeof(buf),`
	`  "cannot initialise '%s' (type %s) with expression of type %s",`
	`  s->var_decl.name, tc_type_name(declared), tc_type_name(init_type));`
	`  tc_error(tc, s->line, buf);`
	`}`

Accepted	The STMT_VAR_DECL structure and error message format — both match my spec exactly. The is_mutable check in STMT_ASSIGN was the right approach and used the symbol table API correctly.
Rejected / Modified	Three things. (1) The AI only checked SYM_ALREADY_DEFINED after scope_define — it did not check SYM_FRAME_FULL. If a scope has more than 64 variables, the variable is silently dropped. I added the missing else-if branch. (2) The STMT_ASSIGN handler used sym->type directly as a TC_Type, but sym->type is WL_Type — a different enum. I added a switch to convert. (3) The error message for assigning to an immutable symbol said 'read-only variable' — I changed it to 'declaration, not a variable (§4.6)' to match the D1 spec language.
Errors you caught	Three, as listed above. I detected (1) by reading the SYM_FRAME_FULL return value documentation in symbol_table.h — the AI had not read that file. I detected (2) by running the type checker on type_error1.wl and getting a compile error: 'comparison between WL_Type and TC_Type.' Bug (1) is the more dangerous one because it is a silent failure — the variable is lost with no error message.
Reflection	E3 confirmed the pattern from Entry 2 (Part 1, Experiment E1): the AI generates structurally correct code but misses cross-file dependencies it has not been shown. Giving it symbol_table.h upfront would have caught the WL_Type/TC_Type confusion. The SYM_FRAME_FULL gap it would probably still have missed because it requires reading the return value documentation carefully. Testing on known inputs immediately is not optional.

Entry 11 of 11 · 2026-05-20 · Testing — D1 consistency review · Claude Sonnet 4.6

Goal	During the final D1 review I noticed that §4.5 said the weight value set is 'non-negative float' but §4.2 defines WEIGHT_LIT as [0-9]+ (integer-only regex). I needed to understand the full consequence and decide which section to fix.
Prompt	WorkoutLang has these two spec sections: §4.2 defines WEIGHT_LIT = [0-9]+("kg"|"lb") — integer only. §4.5 says the weight value set is 'non-negative float paired with unit suffix.' These contradict each other. My lexer follows §4.2: 80kg lexes as WEIGHT_LIT but 3.5kg lexes as FLOAT_LIT 3.5 followed by IDENT kg, which the parser rejects. Which section should I fix and what should it say?
Response (key part)	The AI said: fix §4.5 to say 'non-negative integer.' Its reasoning: the lexer and parser are already submitted and working; changing §4.2 would require changing the lexer with risk of regression. Since the implementation follows §4.2 and all test programs use integer weights, the correct fix is to align §4.5 to match the implementation. It also pointed out that the interpreter stores weight internally as double for the lb→kg conversion, and that this is separate from the syntax — the internal representation can be float even if the literal syntax is integer.
Accepted	The direction: fix §4.5 prose, not §4.2 regex. The internal-storage-vs-literal-syntax distinction was the clearest framing. I also added a sentence documenting that weight values are always printed in kg, so the 135lb → 61.235kg conversion is explicitly specified rather than surprising.
Rejected / Modified	The AI suggested adding a warning that 3.5kg would fail with a confusing parse error. I did not add this — barbell weights in programming contexts are always integer (45lb plates, not 22.5lb). The warning would flag a case that does not arise in the domain. D1 §4.1 explicitly says the language sacrifices generality for the target domain.
Errors you caught	The AI said '3.5kg would produce a lexer error.' It would not — the lexer produces two valid tokens (FLOAT_LIT and IDENT), and the parse error comes from the parser, not the lexer. The distinction matters: the error message says 'expected = for assignment' rather than 'unknown token,' which is why it is confusing. If I had cited 'lexer error' in the spec the exam description would have been wrong.
Reflection	This entry is a good example of the AI being useful for a design consistency question even when the question is about its own output from an earlier session. The key insight — internal representation and literal syntax are independent decisions — was something I had not fully articulated before asking. Writing the prompt forced me to state the contradiction precisely, which is half of solving it.

All entries document substantive sessions with Claude Sonnet 4.6 dated within the Part 2 working period (9–22 May 2026). Entries are ordered chronologically; dates are the actual days on which each session took place.

Claude chat history: https://claude.ai/share/341b49cd-edcc-44ef-98a2-84a94add2f4a